

Scrappy and Substiports: User-Inserted Ad Hoc Objects For Faster, More Sustainable 3D Printing

Ludwig Wilhelm Wall
ludwigwall@gmail.com
School of Computer Science,
University of Waterloo
Waterloo, Canada

Oliver Schneider
oliver.schneider@uwaterloo.ca
Department of Management Science
and Engineering, University of
Waterloo
Waterloo, Canada

Daniel Vogel
dvogel@uwaterloo.ca
School of Computer Science,
University of Waterloo
Waterloo, Canada

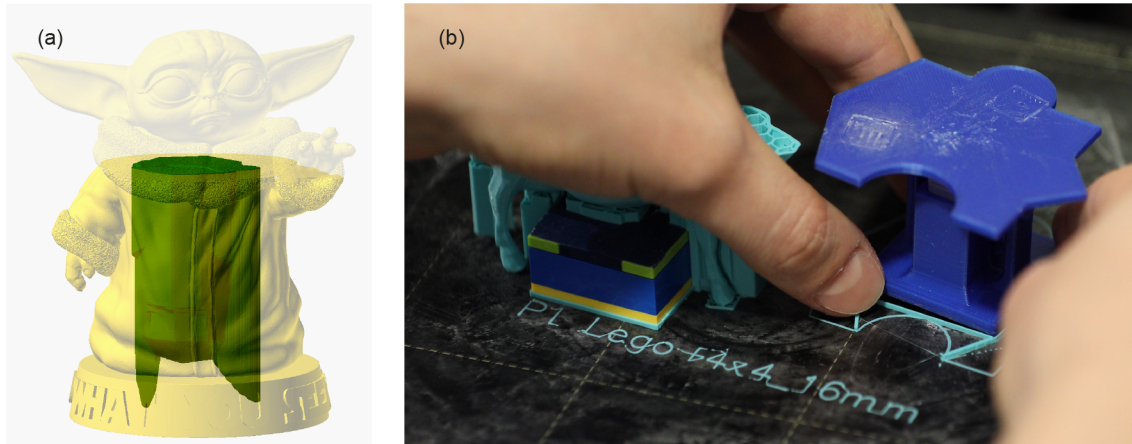


Figure 1: Two types of user interactions within our approach: (a) While modelling, Scrappy visualizes selected scrap that replaces internal support material of Baby Yoda. (b) A user manually places a screw jack in place of external support, guided by instructions included in the G-code by the Substiports slicer.

ABSTRACT

We demonstrate the Scrappy and Substiports systems that use an intermittent interaction approach and integrate into existing CAD software to reduce print time and material with unmodified fused deposition modelling printers. The approach uses ad hoc objects inserted by a user during printing as a replacement for printed internal and external support structures. Examples of objects include household items like books, scrap objects, toy bricks, and custom mechanisms like a screw jack. Our software-only systems assist in maintaining a library of replacement objects and allow customization of the effort to savings trade-off of making printing more sustainable through manual interactions.

CCS CONCEPTS

• **Human-centered computing** → **Interaction techniques.**

KEYWORDS

fabrication, interactive 3D printing, sustainability

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

CHI EA '24, May 11–16, 2024, Honolulu, HI, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0331-7/24/05

<https://doi.org/10.1145/3613905.3648676>

ACM Reference Format:

Ludwig Wilhelm Wall, Oliver Schneider, and Daniel Vogel. 2024. Scrappy and Substiports: User-Inserted Ad Hoc Objects For Faster, More Sustainable 3D Printing. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '24)*, May 11–16, 2024, Honolulu, HI, USA. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3613905.3648676>

1 INTRODUCTION

3D printers that employ fused deposition modelling (FDM) use printable support structures to reliably manufacture models with overhangs, bridges, and other “overhang” geometry extending more than 45° from the Z-axis. Disposable support structures are pure waste, costing additional material and time, so it is desirable to minimize them. One strategy is to optimize the support structure itself, for example, using lightweight structures that branch out from lower parts of the model to support overhangs [3]. However, this introduces additional connections between support material and the model, marring the surface finish. In general, optimizing support geometry increases the chance of failure: thin tall structures are more severely affected by thermal deformation and their low stiffness can result in accidental displacement. For these and similar reasons, support structures typically generated by slicers are sturdy with a substantial connection to the build plate and filling much of the volume beneath overhangs [1].

Similarly, internal support structures may also be required for a successful print. This infill sparsely fills the internal volume, most

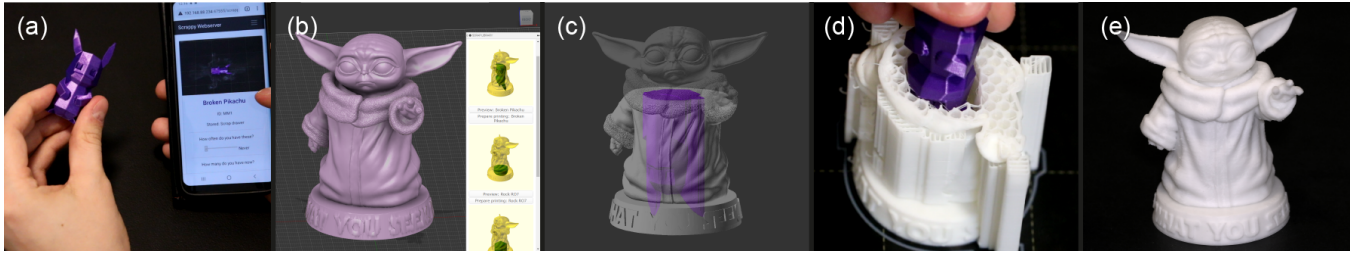


Figure 2: Scrappy system workflow: (a) a library tracks scrap objects, like an old print, as well as other common objects; (b) an add-in for Fusion 360 suggests scraps from the library that can fit into a model, sorted by saved printing time; (c) an algorithm finds the best scrap placement and generates a modified model; (d) a custom slicer optimizes infill and adds machine commands to pause the printer with instructions how to insert the scrap into the object; (e) the final printed object has the scrap inside, reducing the material, time, and energy needed to print.

commonly using a repeating pattern such as a honeycomb. Replacing internal supporting structures can further reduce the material consumption of 3D printing. Our Scrappy [7] and Substiports [8] systems enable replacing internal and external support structures respectively through brief user interactions.

2 THE SCRAPPY SYSTEM

Our Scrappy system automatically searches a library of scrap objects for ones that fit into a CAD model, ordered by how much infill it would replace. The implemented system consists of a Fusion 360 plugin to monitor the modelling process, search for fitting scrap objects in the background, and compute a hollowed 3D model and printing instruction files to fabricate the model with the selected scrap inside (Figure 2). Our insertion engine that handles the search for possible scrap inserts is based on an existing 3D nesting algorithm [4]. A web-based scrap library keeps track of scrap objects, helps users find them in the workshop, and allows them to add new scrap. A customized slicer software generates printer code optimized for scrap insertions. We demonstrate how Scrappy handles common scrap, including previous outdated or broken prints, single use packaging, broken hardware, and many more, showing that scrap-filled objects require less material, time, and energy to fabricate.

Our system design minimizes user effort. It supports the user in maintaining a scrap library, providing simple ways to add new scrap, automatically erasing integrated scrap from the database, and providing guidance for identification and location of binned scrap objects. This effort in return provides cost savings to the user, as well as contributing to more sustainable fabrication.

The server side of the system aims to present the user with an ordered list of scraps that can be safely inserted into the target shape during printing, given a library of scrap models. Solving this geometric problem resembles the “Generalized Matryoshka” algorithm by Jacobson [4]. By using a modified version of Jacobson’s method, the question of safe insertion can be answered as a visibility question, whether any part of the insertion path sticks out of the outer model. This question can be handled on a GPU, allowing for maximal parallelization. Further implementation details are in the Scrappy paper [7].

3 THE SUBSTIPTS SYSTEM

Our Substiports system is implemented primarily as modifications to the open-source slicer Kiri:Moto [2]. In the modified Kiri:Moto slicer user interface, the desired level of interaction density and complexity are adjustable, there are optional scheduling constraints expressed as time periods to avoid inserting objects, and a search quality setting to adjust the process run time. After default print paths are generated for support, our system performs a geometric analysis and optimization to select and position objects from a user-maintained library (Figure 3). We use Particle Swarm Optimization (PSO) [5] to explore the solution space since it offers robust control parameters, is sufficiently efficient, and used in related geometry applications [4, 6, 7]. By leveraging software-based optimization and human dexterity instead of a complex mechanical system, our system is compatible with unmodified FDM 3D printers and it is able to use a broad definition of ad hoc objects as support structures. An object is usable as long as it reliably provides an elevated horizontal surface to print on.

Our method uses the sliced 2D polygons representing the 3D model and support areas to select and position support replacement objects. To optimize computation, a copy of these 2D polygons are simplified using a heuristic method to conservatively remove excess precision. Then, our system proceeds in three steps: (1) separating the support into area clusters; (2) searching for candidate replacement objects and positioning them within each cluster; and (3) combining compatible solutions from each cluster into a list of global solutions. After determining the best global solution, the slicing process continues as normal, generating printing paths for the modified set of 2D polygons to accommodate the replacement objects. Our system further modifies G-code to add object holders, insert pauses with brief messages to alert the user to insert objects, and to make layer adjustments to accommodate arbitrary object heights. Working directly with slices and G-code means our technique is compatible with support generation strategies other than standard support fill, such as cones or trees. Full implementation details are in the Substiports paper [8].

4 USER WALKTHROUGH

Users engage with the systems by maintaining the object library, choosing replacement options during modelling and slicing, and by inserting objects into printed holders during printing pauses.

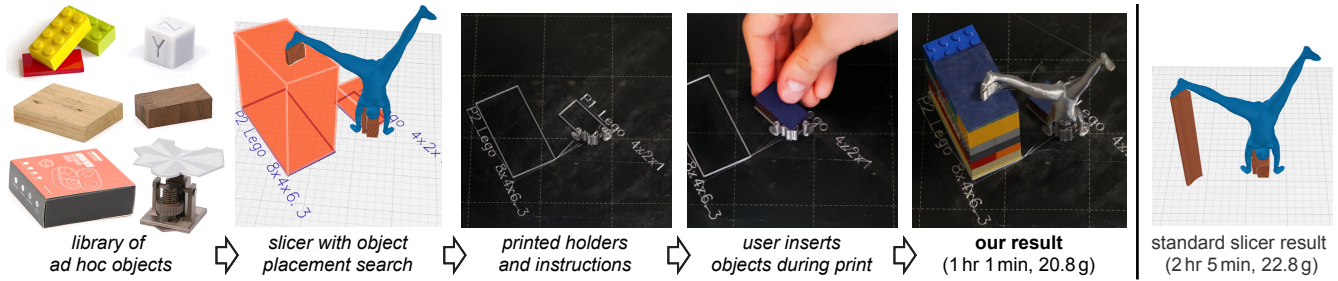


Figure 3: Substiports uses a modified slicer to select reusable objects to use as ad hoc structural support. Object “holders” and instructions are printed, then the user inserts objects during one or more pauses during the print. In this example, our approach reduced print time by 1 hour and saved 87% of support (2.0g out of 2.3g): even a small savings in support material can result in pronounced time savings due to reduced printer movements.

Object Library. A library defines what objects are available to use as support replacements. A base library of standard objects includes items that people may have already, like office supplies and standard packaging, and items often generated when tuning FDM 3D printers, like calibration cubes and certain test models. The user can delete, modify, or add objects to match what they have at hand. Various web-based generator forms provided in Scrapy [7] assist in adding new objects using caliper measurements. Many common objects have a simple shape parameterized by a few easy-to-determine measurements. For example, disposable coffee cups are generally conical and well approximated once the bottom and top radii are measured with a ruler or caliper. A parametric generator can then be used to add the cup to the library. Our library also supports variable objects, like a miniature screw-jack with adjustable height and stackable objects like floppy discs. The building blocks of a construction kit like LEGO can be used as modular ad-hoc replacement objects in addition to or instead of maintaining an object library.

Software Integration. Scrapy integrates directly into the 3D modelling process in the form of a Python Fusion 360 add-in. Using an add-in allows users to benefit from the full modelling capabilities of popular CAD software, while also seamlessly integrating our technique. The add-in creates a new side panel within the Fusion 360 GUI. The panel shows previews of potential scrap able to fit inside the model edited. To gather possible scrap inserts in the first place, the add-in handles exporting the open 3D model, simplifying and potentially repairing it, and sending these files to the server side of our system. Upon receiving scrap insertion options from

the server, the add-in creates preview images and displays those to the user.

To replace external support with support replacement objects using Substiports, the user imports a 3D model into our customized slicer. They select usual slicing settings, with the addition of a new setting for their desired level of involvement during printing. During slicing, the system searches the library for one or more objects to use as support replacements. The slicing result is directly modified to accommodate the replacement objects and rendered showing object positions for the user to inspect.

Inserting Objects. When printing, exported G-code includes commands to generate instructions and “object holders”. When the print reaches the predetermined height where an object must be inserted, the printer pauses, parks the print-head away from the model, and beeps to prompt the user with instructions. Each object is identified by a name and ID printed on the build plate. The user locates the object, then inserts it into the model, or the custom printed holder which ensures it is aligned and secured in place. Depending on object type, the insertion process might differ. For example, a stack of toy bricks may need to be assembled to match a height printed on the build plate (i.e. number of bricks). Similarly, an adjustable model, like a screw jack, may need to be adjusted until it lightly touches the print nozzle (Figure 4b), which our custom G-code positions to the required support height. After inserting one or more objects, the user resumes the print, and this process of intermittent user interactions repeats as needed.

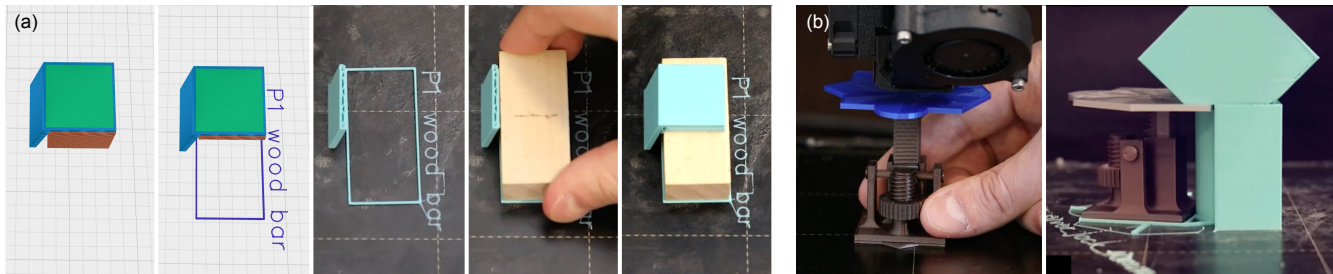


Figure 4: Slicing and insertion examples: (a) Angle Bracket model sliced to use a wood off-cut as a replacement object, the wood is inserted into a “holder” marked with instructions during the print; (b) the screw-jack replacement object is adjusted to match the nozzle height of the parked print head, a screw-jack used as support for the Angled Cylinder model.

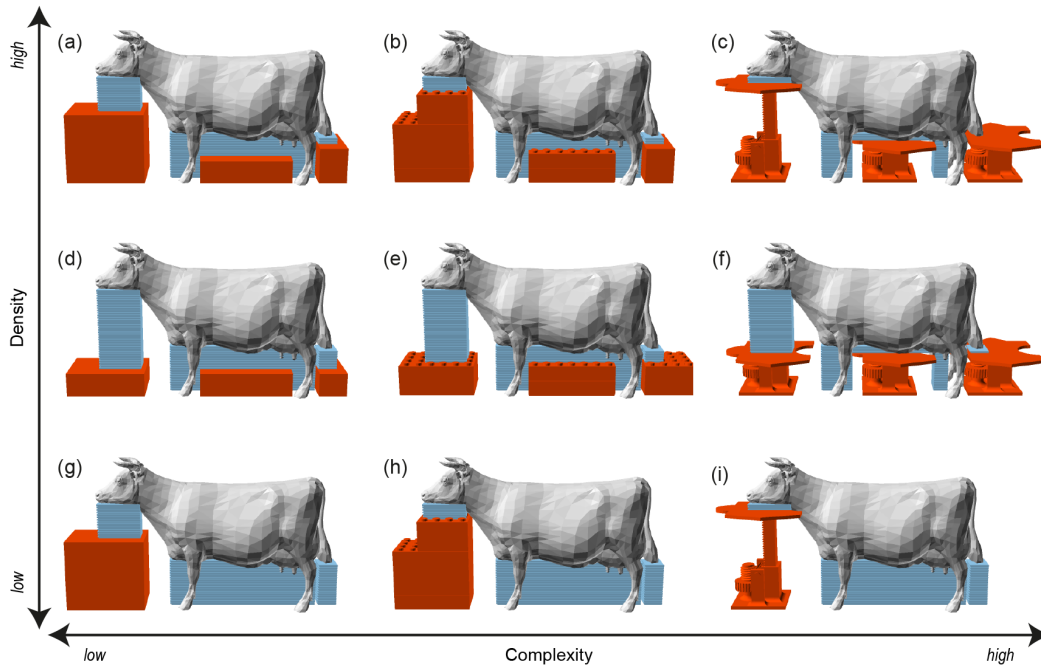


Figure 5: Interaction density and complexity: (g) a single cuboid replacement object requires only a single simple user interaction for insertion; (e) multiple more complex replacement objects with the same height also creates only a single interruption, but it requires more time to locate, assemble and insert multiple stacks of objects; (c) high density and complexity result in multiple user interactions throughout the print process and require additional precise adjustment steps at insertion time.

5 OPTIMIZING FOR INTERMITTENT INTERACTIONS

There are a variety of ways that one or more replacement objects can be inserted into a print. We consider user effort to insert objects as the *density* and *complexity* of interactions. Our technical implementations attempt to balance this expected user effort with material and time savings.

Interaction Density. There can be one or more object insertions each time the printer pauses. Density refers to how frequently the user is interrupted during a print, and how many insertion steps are required during each interruption. The rows in Figure 5 illustrate this dimension.

If the print requires only a single replacement object, then there is only one interruption with a single insertion, making the interaction density low (Figure 5 bottom row). If multiple objects are used as support replacements, and all objects have the same height, there is still a single interruption but with multiple interactions to insert each object (Figure 5 middle row). This increases the density of a single interaction. Common types of uniform ad hoc objects exist, such as wooden off-cuts sharing the same standard thickness. When there are multiple replacement objects with different heights, there will be multiple interruptions creating a high density of interactions during the print (Figure 5 top row). The actual time of the interruption also likely affects perceptions of interaction density: consider object insertions during working hours versus in the middle of the night.

Interaction Complexity. Complexity is an orthogonal dimension to interaction density that captures how difficult, time consuming, or error-prone a single interaction during a single object insertion step is. The columns in Figure 5 illustrate the range of this dimension with three examples.

Examples of low complexity interactions are inserting fixed height replacement objects (Figure 5 left column). Objects only need to be located and inserted into the printed holder. A medium complexity interaction example is stacking multiple objects to achieve a desired support height (Figure 5 middle column). This requires more effort to gather the stacked objects, stack them, and check the height. It is more error-prone since it is possible to misunderstand stacking instructions, or introduce unexpected dimensional height variations due to how well the objects stack. An example of a high complexity interaction is using an adjustable replacement object like a screw jack (Figure 5 right column).

In practice, complexity and density could achieve similar results in terms of support savings. For example, Figure 5a has three pauses each with a single simple insertion, while Figure 5i has a single pause but requires a more complex interaction to set a screw jack height; both achieve similar results. Likewise, Figure 5b and f use varying mixtures of complexity and density to achieve comparable results. This means the user has multiple options to choose based on their goals, constraints, and preferences.

Scheduling for Availability. To avoid delays, user interactions should only be scheduled while users are available to perform them. The

system includes a scheduler to search for solutions that avoid user interactions during certain periods of the day, which can be entered during slicing. The current time is used as the print start time unless entered as a slicing parameter. Scheduling restrictions are built into the system by marking solutions with interactions during unavailable hours as invalid, and by severely reducing their fitness value during optimization.

6 CONCLUSION

We demonstrated practical software-only solutions that enable brief intermittent interactions where people manually insert ad hoc objects to replace internal and external support material during a FDM 3D print. By involving users even a little bit during the print job, 3D printing can be faster and more sustainable.

REFERENCES

- [1] Marc Alexa, Kristian Hildebrand, and Sylvain Lefebvre. 2017. Optimal Discrete Slicing. *ACM Trans. Graph.* 36, 1, Article 12 (jan 2017), 16 pages. <https://doi.org/10.1145/2999536>
- [2] Stewart Allen. 2020. Kiri:Moto: An open-source slicer for Laser cutting, 3D printing, CNC milling . <https://grid.space/kiri/>. [Online; accessed 4-April-2023].
- [3] Ruiliang Feng, Xianda Li, Lin Zhu, Atul Thakur, and Xiangzhi Wei. 2021. An Improved Two-Level Support Structure for Extrusion-Based Additive Manufacturing. *Robotics and Computer-Integrated Manufacturing* 67 (Feb. 2021), 101972. <https://doi.org/10.1016/j.rcim.2020.101972>
- [4] Alec Jacobson. 2017. Generalized Matryoshka: Computational Design of Nesting Objects. *Computer Graphics Forum* 36 (08 2017), 27–35. <https://doi.org/10.1111/cgf.13242>
- [5] J. Kennedy and R. Eberhart. 1995. Particle swarm optimization. In *Proceedings of ICNN'95 - International Conference on Neural Networks*, Vol. 4. 1942–1948 vol.4. <https://doi.org/10.1109/ICNN.1995.488968>
- [6] Kwang Y. Lee and Jong-bae Park. 2006. Application of Particle Swarm Optimization to Economic Dispatch Problem: Advantages and Disadvantages. In *2006 IEEE PES Power Systems Conference and Exposition*. 188–192. <https://doi.org/10.1109/PSCE.2006.296295>
- [7] Ludwig Wilhelm Wall, Alec Jacobson, Daniel Vogel, and Oliver Schneider. 2021. Scrappy: Using Scrap Material as Infill to Make Fabrication More Sustainable. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems* (Yokohama, Japan) (CHI '21). Association for Computing Machinery, New York, NY, USA, Article 665, 12 pages. <https://doi.org/10.1145/3411764.3445187>
- [8] Ludwig Wilhelm Wall, Oliver Schneider, and Daniel Vogel. 2023. Substiports: User-Inserted Ad Hoc Objects as Reusable Structural Support for Unmodified FDM 3D Printers. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology* (San Francisco, CA, USA) (UIST '23). Association for Computing Machinery, New York, NY, USA, Article 34, 20 pages. <https://doi.org/10.1145/3586183.3606718>